

PROJECT DOCUMENTATION

Campus Flow

Smart Campus Companion & Service Request Management System

Project Type

Full-stack web application for role-based campus operations, academic information and campus service request management.

Prepared By: LAKSHYA JAIN

Registration No.: CL20260703011709337

Batch: TECFJ 01

Program: B.Tech CSE — Java Full-Stack Development Program

Table of Contents

No.	Section
1	Project Overview
2	Objectives and Scope
3	Technologies and Development Environment
4	System Architecture and Project Structure
5	Functional Modules and Role-Based Access
6	Database Design and ER Diagram
7	Data Flow Diagrams — Level 0, 1 and 2
8	System Flowchart
9	Frontend Design and Implementation
10	Backend, REST APIs and Security
11	Validation, Testing and Deployment
12	Conclusion and Future Scope

1. Project Overview

CampusFlow is a full-stack web application designed as a smart campus companion for students, faculty and administrators. It combines role-based access, academic information and campus service-request management in a single web portal.

The central purpose is to make campus operations visible and trackable. Students and faculty can raise service requests by specifying the department, location and problem details. Administrators can review those requests and manage their status as Pending, Resolved or Declined.

The portal also provides courses, assignments, notices, events and timetable information. These records are persisted in MongoDB and accessed through Spring Boot REST APIs.

Project Snapshot

Item	CampusFlow
Application Type	Full-stack web application
Frontend	React with Vite, JSX and CSS
Backend	Java Spring Boot REST API
Database	MongoDB
Primary Roles	Student, Faculty, Admin
Authentication	Role-based login with token-based authorization
Core Purpose	Campus service request and academic/campus information management

Core Modules

- User onboarding — student and faculty registration and login.
- Service requests — create, view and track campus service issues.
- Request administration — admin review and status management.
- Academic records — courses and assignments.
- Campus information — notices, events and timetable.
- Profile and role-specific dashboards.
- Light/dark theme and fullscreen controls.

2. Objectives and Scope

- 1. Provide a centralized campus portal for students, faculty and administrators.
- 2. Implement secure role-based access so each user sees appropriate functionality.
- 3. Allow students and faculty to submit structured campus service requests.
- 4. Allow administrators to review requests and update their status.
- 5. Maintain academic and campus records in persistent MongoDB collections.
- 6. Provide a modern React interface with reusable components and theme support.
- 7. Demonstrate full-stack integration between React, Spring Boot REST APIs and MongoDB.

Project Scope

The current scope covers authentication, role-based dashboards, campus service requests, academic/campus records and administrative management. It is designed as a practical full-stack academic application and can be extended with notifications, attachments, analytics and additional campus services.

3. Technologies and Development Environment

Layer	Technology	Purpose
Frontend	React	Component-based UI
Tooling	Vite	Development server/build
Styling	CSS	Layout, responsive design, themes
Backend	Java + Spring Boot	REST API and application logic
Security	Spring Security / token authorization	Authentication and role permissions
Database	MongoDB	Persistent document storage
Integration	Spring Data MongoDB	Repository-based database access
Build	Maven	Backend dependency/build management
Development	VS Code / terminal	Source development and execution

Runtime

Backend development server: port 8080. Vite frontend commonly runs on port 5173.

4. System Architecture and Project Structure

CampusFlow follows a layered full-stack architecture. React handles presentation, client-side state and API calls. Spring Boot receives REST requests, applies security and business logic, and uses Spring Data MongoDB repositories to communicate with MongoDB.

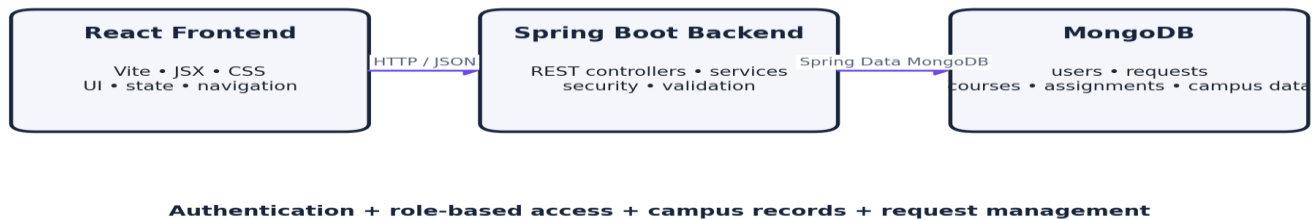


Fig. 1 — CampusFlow high-level system architecture

Backend Project Structure

Folder / File	Responsibility
config	Application and supporting configuration.
controller	REST endpoints consumed by the frontend.
dto	Structured API request/response objects where applicable.
model	MongoDB domain/document model classes.
repository	Spring Data MongoDB repositories.
security	Authentication and authorization components.
service	Business logic between controllers and repositories.
CampusFlowApplication.java	Spring Boot application entry point.
resources/application.properties	Runtime configuration.
pom.xml	Maven dependencies and build configuration.

Frontend Project Structure

File / Folder	Responsibility
src/main.jsx	Main React application, components, state and dashboard UI logic.
src/api.js	Centralized REST API communication using fetch.
src/style.css	Global styling, responsive layout and light/dark themes.
index.html	HTML entry point containing the React root.
package.json	Frontend dependencies and npm scripts.
vite.config.js	Vite configuration.
node_modules	Generated npm dependencies.

5. Functional Modules and Role-Based Access

Student

- Student registration and login.
- Student dashboard and role-specific navigation.
- View courses, assignments, notices, events and timetable.
- Create campus service requests with department, location and problem details.
- Track submitted request status.
- View/update profile where permitted.

Faculty

- Faculty registration and login.
- Faculty-specific dashboard and navigation.
- Access academic and campus information.
- Manage permitted academic/campus records.
- Raise service requests and track status.
- Use capabilities not exposed to students.

Admin

- Highest system authority.
- View/manage registered accounts.
- Review service requests raised by students and faculty.
- Change request status to Pending, Resolved or Declined.
- Manage exposed academic/campus records.
- Monitor overall system information.
- Does not raise normal campus service requests.

Major Record Types

Record	Purpose	Primary Users
Users	Accounts, role and profile	Admin / system
Service Requests	Campus issues and services	Student, Faculty, Admin
Courses	Academic course information	Student, Faculty, Admin
Assignments	Course assignment information	Student, Faculty, Admin
Notices	Announcements	Student, Faculty, Admin
Events	Campus events	Student, Faculty, Admin
Timetable	Class schedules	Student, Faculty, Admin

6. Database Design and ER Diagram

CampusFlow uses MongoDB as its persistent document database. MongoDB stores documents in collections, while Spring Data MongoDB maps Java model classes to those documents and provides repository-based access.

Collection	Representative Data	Purpose
users	name, email, role, department, active	Student, faculty and admin accounts.
service_requests	requesterId, department, location, description, status	Campus request lifecycle.
courses	code, name, department, semester, credits, facultyName	Course catalogue.
assignments	courseId, courseName, title, dueDate, facultyId	Assignment records.
notices	title, content, category, audience, createdBy	Announcements.
events	title, date, venue, description	Campus events.
timetable	day, startTime, endTime, courseId, room	Timetable entries.

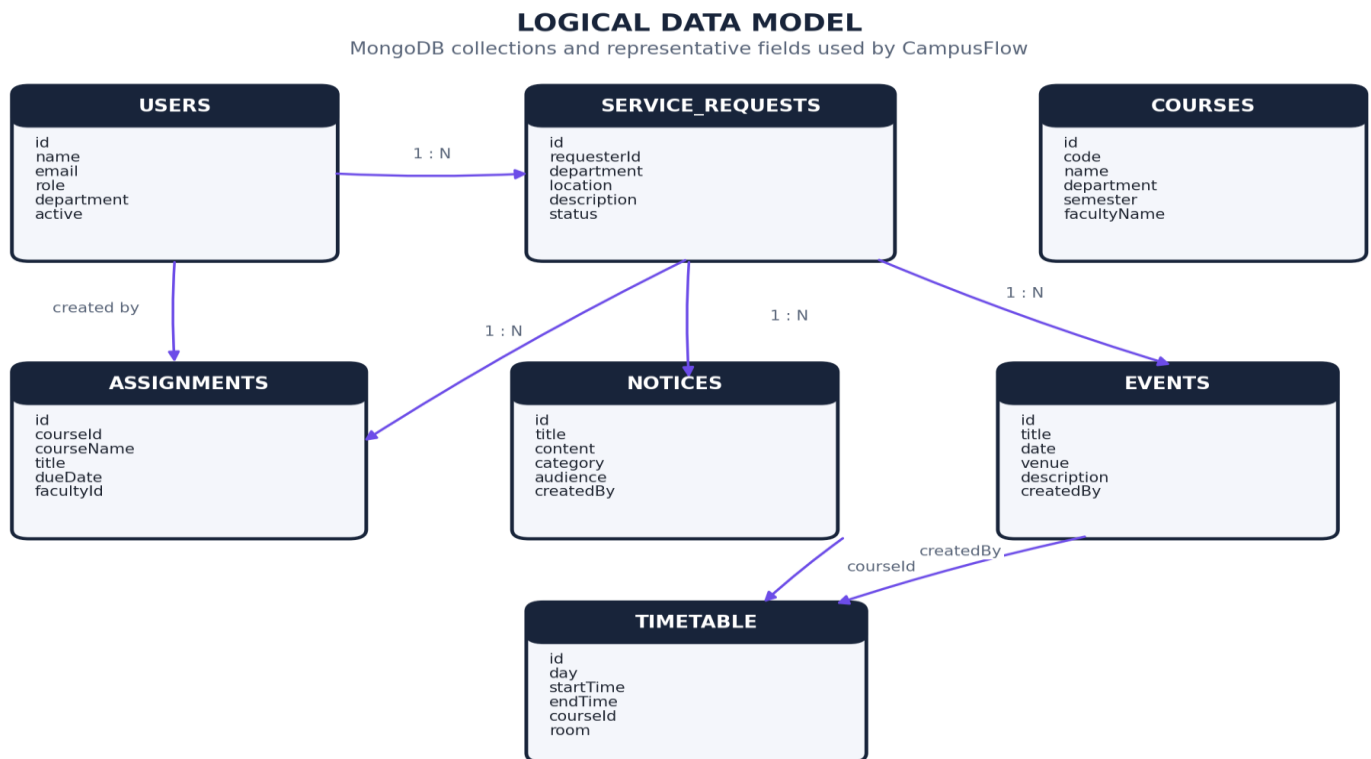


Fig. 2 — Logical ER-style representation of the CampusFlow MongoDB document model

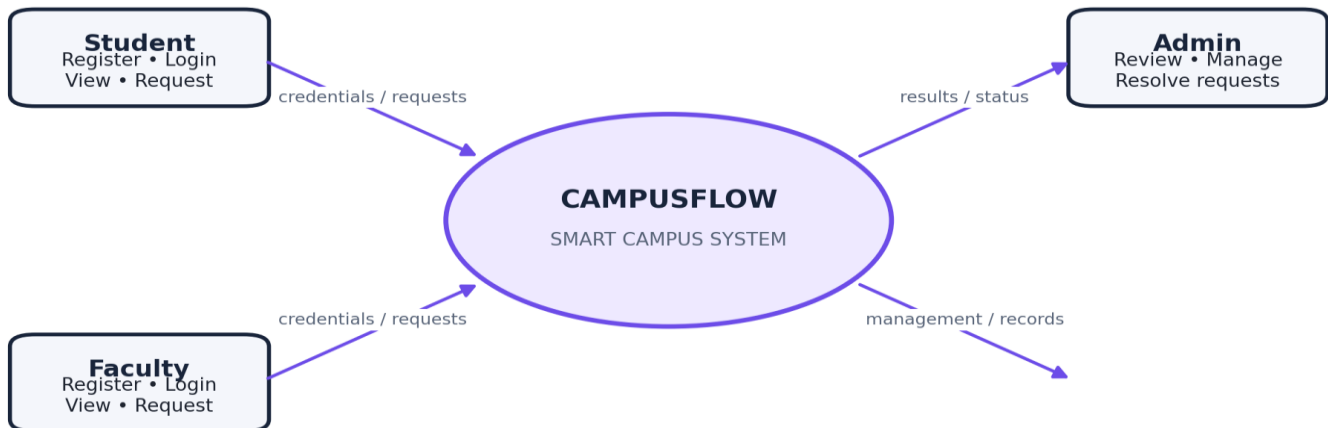
Connectivity

React does not connect directly to MongoDB. Requests travel through Spring Boot REST APIs, which use Spring Data MongoDB repositories.

7. Data Flow Diagrams — Level 0 and Level 1

DFD Level 0 — Context Diagram

At context level, CampusFlow is treated as one system. Students and faculty use it for authentication, campus information and service requests. The administrator uses it for monitoring, record management and request resolution.



Context-level view: external actors exchange information with one CampusFlow process.

Fig. 3 — DFD Level 0: CampusFlow context diagram

DFD Level 1 — Major Processes

Level 1 decomposes the system into authentication, request management and academic/campus record management.

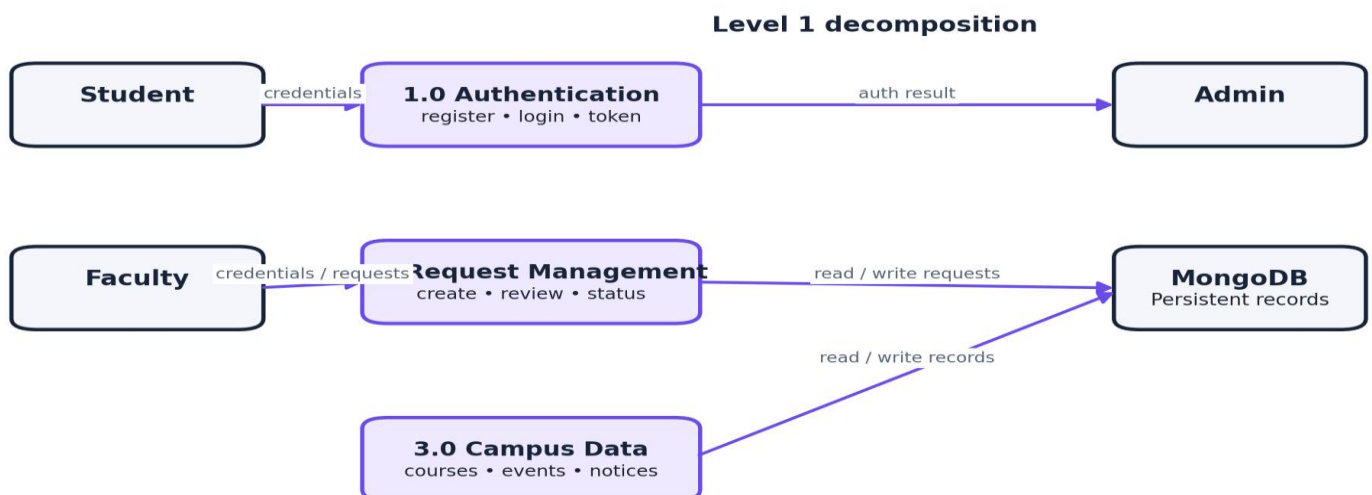


Fig. 4 — DFD Level 1: major CampusFlow processes

7. Data Flow Diagrams — Level 2

Level 2 shows representative sub-processes: validation, authentication/token issuance, request creation/review/status update, and reading/adding campus records.



Fig. 5 — DFD Level 2: representative detailed processes

Request Management Data Flow

1. Student/faculty enters department, location and problem/service description.
2. React validates the input and sends it to Spring Boot.
3. Backend authenticates the user and applies role permissions.
4. Request is stored in MongoDB with an initial status such as Pending.
5. Admin retrieves and reviews the request.
6. Admin updates status to Pending, Resolved or Declined.
7. Updated status is stored and becomes visible to the requester.

8. System Flowchart

The common user journey is shown below. Dashboard options vary by role, but authenticated users pass through the same general entry and authorization stages.

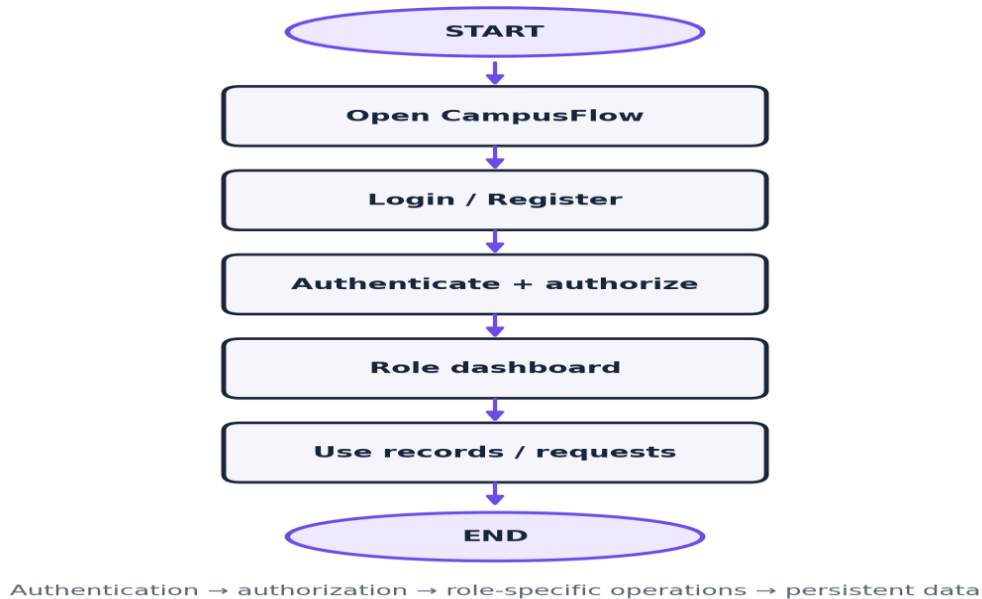


Fig. 6 — CampusFlow overall user flowchart

Request Lifecycle

Lifecycle

Create Request → Pending → Admin Review → Resolved or Declined. The requester can view the resulting status.

Role Decision

- Student → student dashboard and student permissions.
- Faculty → faculty dashboard and extended academic/campus permissions.
- Admin → administrative dashboard and highest management authority.
- Unauthorized users → authentication layer.

9. Frontend Design and Implementation

The CampusFlow frontend is implemented with React and Vite. It uses functional components and React hooks such as `useState` and `useEffect`. Reusable components handle common branding, fields, alerts, navigation, theme controls and data lists.

File	Key Responsibility
main.jsx	Main React application, components, state, navigation and dashboards.
api.js	Central REST API layer; attaches the stored authorization token.
style.css	Global styling, cards, forms, dashboards, buttons, responsive layout and themes.
index.html	HTML document containing the React root.
package.json	NPM scripts and dependencies.
vite.config.js	Vite configuration.

Frontend API Pattern

`api.js` uses a shared request helper. The backend base URL is read from `VITE_API_URL` when available and otherwise defaults to `http://localhost:8080`. If a token exists in `localStorage`, it is sent as a Bearer Authorization header.

Example

Events → `api.events()` → GET `/api/events` → Spring Boot → MongoDB → JSON → React state → Events UI.

User Experience

- Role-specific navigation.
- Light/dark theme support using shared state and CSS variables.
- Fullscreen controls.
- Immediate registration validation.
- Record tabs can fetch current data when opened rather than depending on a browser refresh.

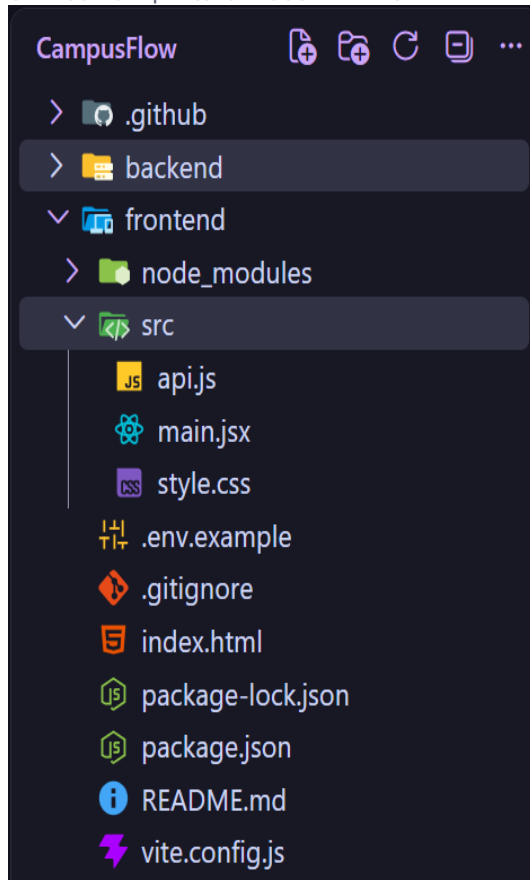


Fig. 7 — Frontend project structure supplied for CampusFlow

10. Backend, REST APIs and Security

The backend is a Spring Boot application run with Maven. It exposes REST APIs consumed by React and follows a layered structure of controllers, services, repositories, models, DTOs and security/configuration.

Layer	Responsibility
Controller	Receives HTTP requests and returns HTTP/JSON responses.
Service	Contains business logic.
Repository	Spring Data MongoDB persistence and CRUD operations.
Model	MongoDB document/domain objects.
DTO	Structured request/response payloads where applicable.
Security	Authentication and role-based authorization.
Config	Application configuration.

Representative REST APIs

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user
POST	/api/auth/register/student	Register student
POST	/api/auth/register/faculty	Register faculty
GET	/api/dashboard	Dashboard data
GET	/api/profile/me	Current profile

PUT	/api/profile/me	Update profile
GET	/api/requests	Request listing
GET	/api/requests/mine	Current user's requests
POST	/api/requests	Create request
PUT	/api/requests/{id}/status	Update request status
GET/POST	/api/events	Read/add events
GET/POST	/api/assignments	Read/add assignments
GET/POST	/api/notices	Read/add notices
GET/POST	/api/courses	Read/add courses
GET/POST	/api/timetable	Read/add timetable

Automated CRUD

Spring Data MongoDB repositories provide standard methods such as `save`, `findAll`, `findById`, `existsById` and `deleteById`. The application exposes only operations appropriate to each role/module rather than granting unrestricted destructive access.

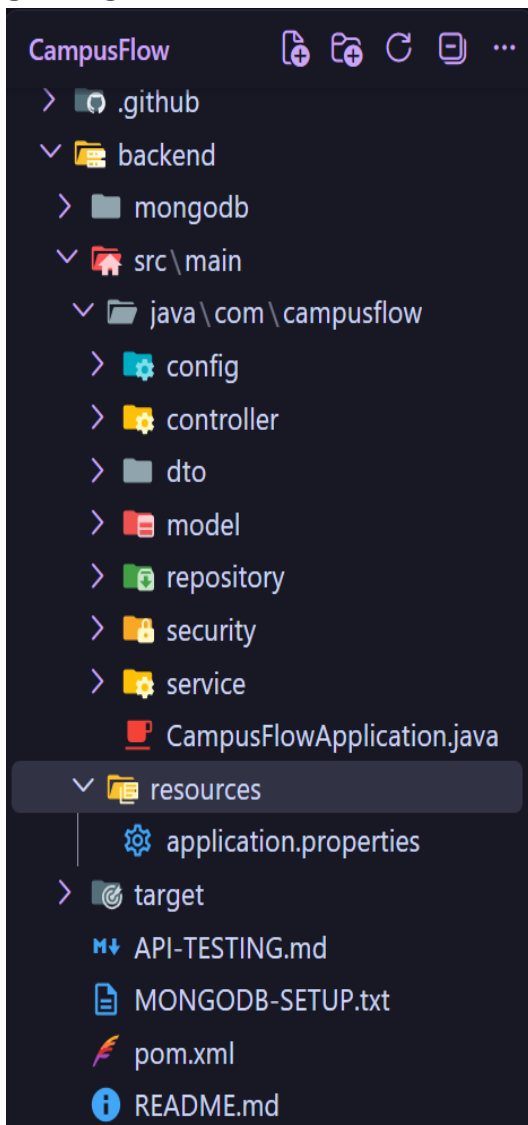


Fig. 8 — Backend project structure supplied for CampusFlow

11. Validation, Testing and Deployment

Input Validation

- Registration fields validate while the user types.
- Names reject arbitrary numeric input and require a valid alphabetic format.
- Email format is checked before submission.
- Password length/basic composition is checked.
- Student/employee identifiers use an acceptable identifier format.
- Required fields prevent incomplete submissions.
- Backend validation remains necessary because client-side validation can be bypassed.

Functional Test Scenarios

Scenario	Expected Result
Valid student registration	Account is created; user can sign in.
Invalid registration name	Immediate message; submission blocked.
Valid login	Authenticated user reaches correct dashboard.
Student/faculty creates request	Request stored and visible to requester.
Admin reviews request	Request visible; status can be managed.
Add event/assignment/notice/course	Record persisted through API and appears in its tab.
Open record tab after DB update	Latest data is fetched without full browser refresh.
Change theme	Light/dark theme switches consistently.
Logout	Client authentication data is cleared and user returns to login.

How to Run

- 1.Start MongoDB and ensure the configured connection is available.
- 2.In backend directory run `mvn spring-boot:run`.
- 3.Confirm Spring Boot starts on port 8080.
- 4.In frontend directory run `npm install` if `node_modules` is absent.
- 5.Run `npm run dev`.
- 6.Open the Vite URL shown in the terminal, typically `http://localhost:5173`.

Connectivity check

A healthy backend startup should show Tomcat on port 8080 and a successful MongoDB driver connection. The React frontend then calls the REST APIs through `api.js`.

12. Conclusion and Future Scope

CampusFlow demonstrates a complete full-stack campus management workflow using React, Spring Boot and MongoDB. It combines role-based access, campus service requests and academic/campus information in one application.

The separation of presentation, API communication, business logic and persistence makes the system understandable and maintainable: React handles the user experience, Spring Boot provides controlled REST endpoints and security, and MongoDB stores persistent records.

Key Outcomes

- Unified portal for three distinct roles.
- Structured service-request lifecycle.
- Persistent academic and campus records.
- Centralized REST communication through api.js.
- Layered Spring Boot backend with repository-based persistence.
- Reusable React UI with theme support.
- Immediate registration validation.

Future Scope

- Email/in-app status notifications.
- File/image attachments for requests.
- Admin analytics and resolution metrics.
- Search, filters and sorting.
- Audit history for administrative changes.
- Fine-grained permissions.
- Production deployment with HTTPS and environment-specific configuration.
- Automated frontend/backend test suites.

Final Summary

CampusFlow

A role-based smart campus platform connecting students, faculty and administrators through one flow of authentication, campus information and service-request management.

End of Project Documentation